

Detailed Research and Implementation Plan for MXINT8 Metadata Execution in Gemmini

Research Planning Document

April 25, 2026

Abstract

This document gives a detailed implementation and publication plan for adding MXINT8 support to GEMMINI with the research focus narrowed to scale-aware metadata execution for transposer-based systolic arrays. The project should not be framed as a broad datatype port. The intended contribution is a metadata-aware memory, control, and execution mechanism that lets a systolic-array accelerator preserve OCP-style microscaling semantics without treating scale bytes as ordinary matrix payload. The plan emphasizes a two-person implementation path: first build a low-risk aligned DIM=32 version, then implement the real research contribution in a default-style DIM=16 configuration where one 32-element MX block spans two physical Gemmini K phases. The document incorporates the critical-review conclusion that the topic is timely and publishable only if framed as a metadata-execution architecture problem rather than a datatype-porting exercise. It covers the algorithm, hardware architecture, software stack, Gemmini extension steps, validation strategy, FireSim simulation plan, ASIC/FPGA industry relevance, paper impact, risks, ablations, and final deliverables.

Contents

1	Executive Decision	2
1.1	Critical-Review Verdict	2
1.2	ASIC and FPGA Industry Relevance	3
1.3	Non-Negotiable Paper Thesis	3
2	Background and Design Constraints	4
2.1	OCP MX and MXINT8 Semantics	4
2.2	Gemmini Constraints	5
3	Formal Problem Statement and Correctness Target	5
4	Algorithm	6
4.1	Matrix and Metadata Layout	6
4.2	Quantization Policy	6
4.3	Reference GEMM Algorithm	7
5	Theoretical Guidelines	8
5.1	Block Partial Bound	8
5.2	Accumulator Exponent Policy	8
5.3	Metadata Is Not Payload	8

6	Hardware Architecture	9
6.1	Top-Level Principle	9
6.2	Architecture Design Guidelines from the Critical Review	9
6.3	Main Hardware Blocks	10
6.3.1	MXScaleSRAM	10
6.3.2	MXKBlockController	10
6.3.3	ScaleVectorGenerator	11
6.3.4	BlockScaleUnit	11
6.4	Accumulator Strategy	12
7	Software Needed	12
7.1	Software Components	12
7.2	Software Descriptor	13
7.3	Runtime API	13
8	Steps for Gemmini Extension	14
8.1	Step 0: Freeze Scope and Repository State	14
8.2	Step 1: Golden Model and Packer	14
8.3	Step 2: Add Configuration Parameters	15
8.4	Step 3: Add MXScaleSRAM	15
8.5	Step 4: Add Config and Command Interface	15
8.6	Step 5: Add MXKBlockController for DIM=32	16
8.7	Step 6: Add BlockScaleUnit for K=32	16
8.8	Step 7: Support K Greater Than 32	17
8.9	Step 8: Implement DIM=16 Two-Phase K-Block Execution	17
8.10	Step 9: Add Transposer-Aware ScaleVectorGenerator	17
8.11	Step 10: Integrate Loop-Level Support	17
9	Validation of the Designed Architecture	18
9.1	Validation Layers	18
9.2	Required Corner Cases	18
9.3	Invariants to Assert	19
9.4	Debugging Strategy	19
10	FireSim Simulation Plan	19
10.1	When to Use FireSim	19
10.2	FireSim Setup Checklist	20
10.3	Counters to Add	20
10.4	FireSim Experiments	20
11	Baselines, Ablations, and Evaluation Discipline	21
12	Paper Impact	22
12.1	What the Paper Contributes	22
12.2	What Not to Claim	22
12.3	Paper Figures	23
12.4	Paper Tables	23

13 Risk Management	24
13.1 Critical Weaknesses to Fix Before Submission	24
13.2 Main Risks	24
13.3 Stop/Go Criteria	25
14 Two-Person Work Plan	25
15 Publication Roadmap and Milestones	25
16 Recommended Final Deliverables	26
17 One-Page Execution Checklist	27
17.1 Before More RTL	27
17.2 Hardware Must-Haves	27
17.3 Evaluation Must-Haves	28
18 Final Recommendation	28

1 Executive Decision

The project should focus on **MXINT8 metadata execution**, not on MXFP8, MXFP4, or a family of MX formats. This is the lowest-risk path to a real architecture contribution. MXINT8 preserves the central microscaling problem: each block has an 8-bit shared scale and 32 private 8-bit payload elements. However, unlike MXFP8, it lets the first hardware version reuse Gemmini’s existing int8 multiply-accumulate machinery. The hard part becomes metadata alignment, K-block tracking, transposer interaction, and correct scaled accumulation. That is exactly the part that can be made publishable.

The core claim should be:

Microscaling is not plug-and-play for transposer-based systolic arrays. In Gemmini-like accelerators, MX metadata must be tracked by logical 32-element K-blocks rather than by physical scratchpad rows or transposer cycles. A sidecar metadata memory, logical K-block controller, and block-scale application stage preserve MXINT8 semantics while avoiding duplicate transposed storage and unnecessary requantization.

The implementation should proceed in two stages:

1. **Stage A: DIM=32 aligned MVP.** One Gemmini K episode corresponds naturally to one 32-element MX block. This derisks the metadata SRAM, software packer, command interface, and block-scale logic.
2. **Stage B: DIM=16 scale-aware research design.** One logical MX block spans two physical Gemmini K phases. This is where the paper’s real contribution lives.

The final paper should not say “we added MXINT8 to Gemmini.” It should say “we designed and evaluated a logical MX metadata execution mechanism for transposer-based systolic arrays, using Gemmini as the full-stack implementation vehicle.”

1.1 Critical-Review Verdict

The research topic is **good**, but the current paper framing must be disciplined. A paper whose main claim is “we added MXINT8 to Gemmini” will look like an implementation note. A paper whose main claim is “we formalized and solved metadata execution for block-scaled numerics when the logical MX block size and physical systolic-array width do not match” is a real architecture paper.

The decision table for the project is:

Dimension	Verdict	Required action
Technical direction	Strong	Keep MXINT8 as the vehicle, but sell logical metadata execution as the contribution
Current publishability	Not yet	Add formal semantics, baselines, workload definitions, and area/timing evidence
ASIC relevance	High	Tie the work to current microscaling and block-scaled commercial accelerators
FPGA relevance	Moderate	Position as forward-looking architecture research, not as a current mainstream FPGA software-flow feature
One-year feasibility	Feasible	Freeze semantics immediately; finish DIM=32 by month 4 and DIM=16 by month 6

The paper should be written as a conference-first project. A journal extension should only follow after broader dataflow support, stronger synthesis results, and a more general argument about metadata execution in block-scaled accelerator generators.

1.2 ASIC and FPGA Industry Relevance

The industrial motivation is real. The Open Compute Project standardized MX formats, including integer and floating microscaling formats, and explicitly positions microscaling as a low-precision path for efficient AI computation [1]. Commercial ASIC accelerators are also moving toward block-scaled and microscaled formats: NVIDIA Blackwell Tensor Cores advertise community-defined microscaling support, NVIDIA Transformer Engine and cuDNN expose Blackwell microscaling paths, and AMD advertises MX-style low-precision formats and software quantization flows in current Instinct and Quark materials [6, 7, 8, 9, 10]. That makes the ASIC side of this topic timely.

The FPGA story is more nuanced. AMD and Intel FPGA/NPU deployment documentation still emphasizes practical INT8, BF16, FP16, and mixed-precision flows more than native MXINT8 deployment [11, 12]. Therefore, the paper should not claim that MXINT8 is already a mainstream FPGA product feature. The correct claim is that FPGA and open accelerator generators are excellent research vehicles for studying metadata-aware execution mechanisms that commercial ASICs increasingly need.

This distinction matters for reviewers. The project is industry-relevant because block-scaled numerics are becoming commercially important; it is publishable because open Gemini/FireSim infrastructure can expose the microarchitectural cost of metadata alignment, transposition, and logical/physical K-block mismatch. The paper must not overclaim direct product adoption on current FPGA toolchains.

1.3 Non-Negotiable Paper Thesis

The final paper thesis should be written approximately as:

Block-scaled tensor formats introduce metadata whose logical reduction domain is independent of the physical episode width of a systolic array. When the MX block size is 32 but the accelerator executes 16-wide K phases, correct execution requires logical K-block tracking, sidecar metadata delivery, and block-boundary scaling before cross-block accumulation. We implement and evaluate this mechanism in Gemini and show its cost relative to software-repack, duplicate-metadata, and no-stall baselines.

This thesis is broader than Gemmini and narrower than an unrealistic “support all MX formats” claim. That is the right scope for two PhD researchers in one year.

2 Background and Design Constraints

2.1 OCP MX and MXINT8 Semantics

OCP microscaling formats represent a block of values using a shared scale and a fixed number of low-precision payload elements. For the concrete MX formats relevant here, the block size is

$$k = 32.$$

Each block has one E8M0 shared scale and 32 private payload elements. In MXINT8, the payload elements are signed 8-bit integers. Conceptually, for payload element p_i and block scale X ,

$$v_i = X \cdot p_i.$$

For a dot product between an A block and a B block, the shared scales factor out of the inner reduction:

$$C_{ij}^{(b)} = X_{i,b}^A X_{b,j}^B \sum_{t=0}^{31} p_{i,b,t}^A p_{b,t,j}^B.$$

For a full dot product over multiple MX blocks,

$$C_{ij} = \sum_b C_{ij}^{(b)}.$$

The most important correctness rule is therefore:

For $K > 32$, the scale product must be applied once per logical 32-element K-block before adding that block’s contribution to the final accumulator. It is incorrect to accumulate all raw int8 products across the entire K dimension and apply one scale at the end.

The scale is an E8M0 byte. For ordinary finite scale bytes, the scale can be interpreted as a power of two:

$$X = 2^{s-127}.$$

The implementation should preserve the external scale byte in memory, but internally it can store the signed exponent

$$e = s - 127.$$

Then the block contribution becomes

$$C_{ij}^{(b)} = 2^{e_{i,b}^A + e_{b,j}^B} P_{ij}^{(b)},$$

where

$$P_{ij}^{(b)} = \sum_{t=0}^{31} p_{i,b,t}^A p_{b,t,j}^B.$$

2.2 Gemmini Constraints

Gemmini is row-addressed: scratchpad rows are DIM elements wide, and DIM is tied to the width of the systolic array. In the public/default-style configuration, DIM is 16. Gemmini’s scratchpad uses `inputType`, which is commonly `int8` in the default configuration, and the accumulator uses `accType`, commonly `int32`. Gemmini’s `ExecuteController` instantiates `MeshWithDelays`, which contains the systolic array, a transposer, and delay registers. The transposer is not an optional afterthought; it is part of the normal execution path, especially in output-stationary mode.

This creates the central mismatch:

Property	OCP MX	Default-style Gemmini
Logical scaling block	32 K elements	16-wide K episode
Payload storage	element matrix plus scale metadata	row-addressed scratchpad
Transpose effect	changes scale grouping axis	transposer in compute path
Correct execution	scale per 32-K block	compute in DIM episodes

For DIM=32, one Gemmini K episode can correspond to one MX block. For DIM=16, one MX block spans two Gemmini K phases. The architecture must track the logical 32-element MX block independent of the physical row width.

3 Formal Problem Statement and Correctness Target

This section should become the mathematical core of the paper. Without it, reviewers will see the work as engineering integration rather than architecture research.

Let the MX block size be $B = 32$ and the physical systolic-array inner dimension be $d \in \{16, 32\}$. Let $\hat{a}_{i,b,t}$ and $\hat{b}_{b,t,j}$ denote `int8` payloads within logical MX block b and local K offset t . Let $e_{i,b}^A$ and $e_{b,j}^B$ be decoded shared exponents. The required result is

$$\hat{C}_{ij} = \sum_{b=0}^{\lceil K/B \rceil - 1} 2^{e_{i,b}^A + e_{b,j}^B} \sum_{t=0}^{B-1} \hat{a}_{i,b,t} \hat{b}_{b,t,j}.$$

For a physical execution sliced into episodes of width d , define the logical K-block index of K coordinate k as

$$\beta(k) = \left\lfloor \frac{k}{B} \right\rfloor.$$

The hardware correctness invariant is that all physical episodes whose K coordinates map to the same $\beta(k)$ must use the same A and B scale vectors, and the raw partial corresponding to that full logical block must be scaled before it is merged with any partial from a different logical block:

$$\beta(k_1) = \beta(k_2) = b \Rightarrow \mu_A(i, k_1) = \mu_A(i, k_2), \quad \mu_B(j, k_1) = \mu_B(j, k_2).$$

For $d = 16$, this means the scale vectors are held constant across two physical K phases. For $d = 32$, the invariant is naturally satisfied by one physical episode per logical MX block, but the implementation should still use the same logical metadata interface so that DIM=32 is a bring-up point rather than a separate architecture.

The paper should test three hypotheses:

1. **H1:** Sidecar metadata execution reduces metadata-related data movement or stalls relative to software-repack and duplicate-transposed-metadata baselines.

2. **H2:** Logical K-block tracking preserves exact MXINT8 semantics for $d = 16$ with bounded control, timing, and storage overhead.
3. **H3:** The result persists on realistic GEMM traces, not only on square synthetic matrices.

The minimum empirical thresholds for a conference-quality result are:

- exact agreement with the golden model on deterministic and randomized tests;
- resource/Fmax overhead that is bounded and honestly reported;
- at least one meaningful win over a non-strawman metadata baseline;
- DIM=16 results, not only DIM=32 results.

4 Algorithm

4.1 Matrix and Metadata Layout

For a matrix multiplication

$$C = AB,$$

where

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the MXINT8 representation should use the following logical metadata layout:

$$\begin{aligned} A_{\text{payload}} &\in \mathbb{Z}_8^{M \times K}, & A_{\text{scale}} &\in \mathbb{B}^{M \times \lceil K/32 \rceil}, \\ B_{\text{payload}} &\in \mathbb{Z}_8^{K \times N}, & B_{\text{scale}} &\in \mathbb{B}^{\lceil K/32 \rceil \times N}, \end{aligned}$$

where $\mathbb{B}$ denotes an 8-bit E8M0 scale byte.

For output tile (m_0, n_0) and logical K-block b , the hardware needs:

$$\begin{aligned} \text{aScaleVec}[i] &= A_{\text{scale}}[m_0 + i, b], \\ \text{bScaleVec}[j] &= B_{\text{scale}}[b, n_0 + j]. \end{aligned}$$

The payload matrices are ordinary int8 matrices. The scale matrices are sidecar metadata. The transposer sees payload only; it never sees scale bytes as data.

4.2 Quantization Policy

The quantization policy must be frozen before RTL work begins. A recommended first policy is symmetric block quantization:

1. For each 32-element block, compute

$$a_{\max} = \max_i |x_i|.$$

2. Select a power-of-two scale $X = 2^e$ so that the largest magnitude value maps into the signed int8 payload range under the chosen saturation rule.

3. Quantize each element as

$$p_i = \text{sat}_8(\text{round}(x_i/X)).$$

4. Store the E8M0 byte corresponding to $e + 127$.

The exact signed range must be fixed in the software reference and RTL. There are two reasonable choices:

- **Full two's-complement range:** $[-128, 127]$.
- **Symmetric range:** $[-127, 127]$.

For a first research artifact, the symmetric range is easier to reason about and reduces one asymmetric corner case. However, if strict OCP conformance requires a different interpretation for MXINT8, the implementation must follow the specification. The document, code, tests, and paper must all use the same rule.

The following policy decisions must be written in a versioned file, for example `mxint8_policy.md`:

Decision	Required project choice
Payload range	$[-127, 127]$ or $[-128, 127]$, fixed globally
Rounding	nearest-even recommended for spec alignment
Overflow	saturate to chosen payload range
All-zero block	canonical scale byte plus zero payloads
Tail block	pad invalid K lanes with zero payloads; metadata still present
NaN scale byte	reject in integer-only prototype or define poison behavior
Accumulator output	fixed-point scaled int or software-converted reference output

4.3 Reference GEMM Algorithm

The reference model should implement MXINT8 GEMM by logical K-blocks:

Listing 1: Reference MXINT8 GEMM structure.

```

for (int m = 0; m < M; m++) {
    for (int n = 0; n < N; n++) {
        AccumType acc = 0;

        for (int b = 0; b < ceil_div(K, 32); b++) {
            int32_t raw = 0;
            int valid_k = min(32, K - 32*b);

            for (int t = 0; t < valid_k; t++) {
                int8_t a = A_payload[m][32*b + t];
                int8_t bval = B_payload[32*b + t][n];
                raw += (int32_t)a * (int32_t)bval;
            }

            int eA = decode_e8m0_to_exp(A_scale[m][b]);
            int eB = decode_e8m0_to_exp(B_scale[b][n]);
            acc += scale_block_partial(raw, eA + eB);
        }
    }
}

```

```

    C[m][n] = finalize(acc);
}
}

```

This reference structure must be reflected in hardware: raw int8 products can accumulate within one 32-element MX block, but each block’s raw partial sum must be scaled before being merged with contributions from other blocks.

5 Theoretical Guidelines

The numerical contract must be frozen before RTL. The project should not allow each test or software path to quietly choose its own rounding, exponent, or saturation rule. The following rules are recommended.

5.1 Block Partial Bound

For one logical MXINT8 block with symmetric payload range $[-127, 127]$, the raw dot-product magnitude is bounded by

$$|P_{ij}^{(b)}| \leq 32 \cdot 127^2 = 516,128,$$

which fits easily in signed int32. Therefore, overflow inside a single raw block partial is not the primary risk. The dangerous point is cross-block accumulation after exponent alignment. The implementation should use a 48-bit or 64-bit shadow accumulator internally unless the paper explicitly restricts exponent ranges.

5.2 Accumulator Exponent Policy

The paper must define one of the following policies and test it exhaustively:

Policy	Description	Recommendation
Fixed tile exponent	Choose one accumulator exponent per output tile before accumulation	Best first implementation
Dynamic tile exponent	Update the tile exponent as blocks arrive	More flexible but complicates timing and verification
Wide exact software accumulation	Use software or simulation to preserve exact scaled value	Good for reference only
FP/BF accumulator	Convert each block partial into FP32/BF16	Too large for year-one scope

The hardware design should predecode E8M0 scale bytes into signed exponents when metadata enters MXScaleSRAM. This removes avoidable decode logic from the hottest execution path.

5.3 Metadata Is Not Payload

The sidecar metadata path is not an optimization detail; it is the central architectural idea. Scale bytes should not be inserted into scratchpad payload rows, moved through the payload transposer, or treated as extra matrix elements. Metadata has different indexing, reuse, and correctness rules

than payload. This is why the paper should compare against duplicate-metadata and software-repack baselines rather than only against stock INT8.

6 Hardware Architecture

6.1 Top-Level Principle

The architecture should implement a shadow metadata pipeline:

Stream	Path	Function
Payload	Scratchpad $\rightarrow$ Transposer $\rightarrow$ Mesh	int8 matrix
Metadata	<code>MXScaleSRAM</code> $\rightarrow$ <code>MXKBlockController</code> $\rightarrow$ <code>ScaleVectorGenerator</code> $\rightarrow$ <code>BlockScaleUnit</code>	scale vector

Scales must not be inserted into scratchpad rows as if they were payload. Scales must not be physically transposed through Gemmini’s payload transposer. The metadata path is sideband, logical, and controlled by K-block counters.

6.2 Architecture Design Guidelines from the Critical Review

The recommended architecture should obey the following design rules:

Decision	Recommended choice	Reason
Numerical policy	nearest-even, saturating arithmetic, finite E8M0 scale bytes in year one	One unambiguous reference model
MVP mode	GEMM, weight-stationary first	Keeps the schedule realistic
Final claim scope	WS plus at least one validated transpose/OS case, or explicitly narrow the claim	Prevents reviewer attack on unsupported modes
Internal accumulator	48-bit or 64-bit shadow accumulator	Avoids numerical failure for $K > 32$
Metadata representation	predecoded signed exponents in <code>MXScaleSRAM</code>	Keeps decode off the critical path
Bring-up geometry	DIM=32 first, DIM=16 second	Separates plumbing from the contribution
Year-one exclusions	MXFP8, MXFP4, MXFP6, training, full compiler automation	Avoids uncontrolled scope expansion

A robust timing-friendly implementation should strongly consider a two-step block-scaling path: the mesh first produces a raw int32 partial tile for one logical MX block; a temporary tile buffer holds the raw partial; `BlockScaleUnit` aligns it to the accumulator exponent; and a wide shadow accumulator merges it with previous logical blocks. This may look less elegant than a pure stream, but it is more likely to close timing on FPGA and to produce credible resource/Fmax evidence.

6.3 Main Hardware Blocks

6.3.1 MXScaleSRAM

MXScaleSRAM stores sidcar metadata. It should support:

- write path from DMA/load controller or an `mvin3`-style stream,
- read path for A scale vector,
- read path for B scale vector,
- optional descriptor bits for role, orientation, valid K lanes, and format,
- counters for metadata reads, writes, stalls, and bytes moved.

For DIM=32, one A tile and one B tile need:

$$32 \text{ A scale bytes} + 32 \text{ B scale bytes} = 64 \text{ metadata bytes}$$

per 32-wide logical K-block. Each 32-by-32 int8 tile has 1024 payload bytes, so one side scale row per tile is a small overhead. The overhead is not the research problem; the research problem is keeping that metadata aligned correctly when payloads are tiled, transposed, and split across physical K phases.

6.3.2 MXKBlockController

MXKBlockController is the central control block. It tracks:

- current output tile (m_{tile}, n_{tile}) ,
- current logical K-block b ,
- current physical K phase within the logical block,
- active A scale vector,
- active B scale vector,
- tail mask for the final K-block,
- transpose and dataflow mode bits.

For DIM=32:

Listing 2: DIM=32 logical behavior.

```
fetch A scale vector for logical K-block b
fetch B scale vector for logical K-block b
run one 32-wide Gemini compute episode
scale block output
advance b
```

For DIM=16:

Listing 3: DIM=16 logical behavior.

```

fetch A scale vector for logical K-block b
fetch B scale vector for logical K-block b
run physical K half 0: lanes 0..15
run physical K half 1: lanes 16..31
combine raw partials for the full logical 32-lane block
scale block output once
advance b

```

The non-negotiable invariant is:

For DIM=16, `aScaleVec` and `bScaleVec` remain constant across both 16-wide physical K phases belonging to the same logical 32-element MX block.

6.3.3 ScaleVectorGenerator

`ScaleVectorGenerator` converts stored scale rows into the vectors needed by the current logical operand view. In the MVP, software should prepack metadata in the same logical view that hardware expects. Then `ScaleVectorGenerator` can be almost a pass-through.

In the research version, `ScaleVectorGenerator` must support:

- A and B operand role distinction,
- logical orientation: normal or transposed,
- dataflow-specific interpretation,
- optional permutation or swapping of A/B scale vectors under transpose flags.

The transposer itself should remain mostly unchanged. The scale-vector generator should handle metadata reconstruction, not by streaming scale bytes through another physical transposer, but by mapping logical tile coordinates to vector indices.

6.3.4 BlockScaleUnit

`BlockScaleUnit` consumes raw block partial sums and the scale vectors. For each output row i and column j of the current tile:

$$C_{ij}^{(b)} = 2^{e_i^A + e_j^B} P_{ij}^{(b)}.$$

For MXINT8, the raw partial $P_{ij}^{(b)}$ is int32. The scale product is a power-of-two adjustment. The recommended first implementation is fixed-point exponent alignment rather than full floating-point accumulation.

A practical fixed-point design:

1. Compute raw int32 partial tile for one logical MX block.
2. Compute scale exponent

$$e_{ij} = e_i^A + e_j^B.$$

3. Align to a chosen accumulator exponent E_{acc} :

$$shift_{ij} = e_{ij} - E_{acc}.$$

4. Shift raw partial up or down with saturation/rounding:

$$aligned_{ij} = \text{round_shift}(P_{ij}^{(b)}, shift_{ij}).$$

5. Accumulate aligned values into a wider accumulator, preferably int48 or int64 in the reference design. If Gemmini’s accumulator must remain int32, restrict dynamic range and document the limitation.

The first hardware prototype can use a conservative common exponent per output tile. A more advanced design can compute tile-level output scale dynamically or export block-scaled outputs for software finalization.

6.4 Accumulator Strategy

This is the most dangerous design choice. There are three options.

Option	Benefit	Problem
Fixed-point accumulator	Reuses integer-oriented path; low hardware cost	Requires careful exponent policy and saturation rules
FP32/BF16 accumulator	Clean numerical model	Turns the project into a floating-point datapath project
K=32 only output scaling	Easy bring-up	Not enough for real workloads or paper claims

Recommended path:

1. Bring up K=32 only with output scaling.
2. Add K>32 fixed-point block accumulation.
3. Keep FP accumulation as a later optional extension, not as the first architecture.

7 Software Needed

7.1 Software Components

The software stack must include:

1. **Golden model:** Python or C++ implementation of MXINT8 encode/decode, GEMM, tail handling, and transpose-aware metadata.
2. **Packer:** converts dense int8 or FP32 tensors into payload arrays plus sidecar scale arrays.
3. **Descriptor generator:** emits tile descriptors for payload and scale locations.
4. **Baremetal tests:** small deterministic tests for `mvin`, metadata load, scale read, and MX GEMM.
5. **Gemmini C API extensions:** wrappers in `gemmini.h` or nearby runtime headers.
6. **Benchmark harness:** microbenchmarks and selected end-to-end kernels.

7. **Result checker:** compares hardware outputs to the golden model with exact or tolerance-based rules.
8. **Plot scripts:** produce paper figures from CSV logs.

7.2 Software Descriptor

A minimal software-visible descriptor should look like:

Listing 4: Proposed software MX tile descriptor.

```
typedef struct {
    uint64_t dram_payload;    // DRAM pointer to payload tile
    uint64_t dram_scale;     // DRAM pointer to scale vector/row
    uint32_t sp_payload;     // Gemmini scratchpad base row
    uint32_t sp_scale;       // MXScaleSRAM base row/index
    uint16_t m_tile;
    uint16_t n_tile;
    uint16_t k_block;
    uint8_t  role;           // 0=A, 1=B
    uint8_t  orientation;    // 0=normal, 1=logical-transposed
    uint8_t  valid_k;        // 1..32 for tail block
    uint8_t  flags;
} mxint8_tile_desc_t;
```

Hardware should not depend on all fields initially. The first version should use the descriptor as a software contract. Hardware should need only the scale base, current K-block, valid K lanes, and mode bits.

7.3 Runtime API

Recommended API additions:

Listing 5: Sketch of runtime API.

```
void gemmini_config_mxint8(
    uint32_t mx_enable,
    uint32_t block_size,
    uint32_t scale_base_a,
    uint32_t scale_base_b,
    uint32_t flags);

void gemmini_mvin_mx_scale(
    const uint8_t *dram_scale,
    uint32_t mxscale_addr,
    size_t bytes);

void gemmini_preload_mxint8(
    uint32_t b_payload_sp_addr,
    uint32_t b_scale_addr,
    uint32_t c_acc_addr,
    uint32_t flags);

void gemmini_compute_mxint8(
    uint32_t a_payload_sp_addr,
```

```
uint32_t a_scale_addr,  
uint32_t k_block,  
uint32_t valid_k,  
uint32_t flags);
```

The exact ISA encoding can be decided later. The runtime API should be written first so tests and hardware development converge on the same contract.

8 Steps for Gemmini Extension

8.1 Step 0: Freeze Scope and Repository State

Before coding:

- choose MXINT8 only,
- choose weight-stationary only for MVP,
- choose DIM=32 first,
- freeze Chipyard, Gemmini, FireSim, and toolchain commits,
- write `mxint8_policy.md`,
- write a two-page architecture note.

Exit criterion: both implementers agree on the exact semantics and benchmark plan.

8.2 Step 1: Golden Model and Packer

Implement:

- `encode_mxint8_block()`,
- `decode_mxint8_block()`,
- `pack_A_rowwise()`,
- `pack_B_columnwise()`,
- `mxint8_gemm_reference()`,
- `transpose_aware_metadata_model()`,
- random test generator.

Exit criterion: software reference passes randomized tests for shapes where M , N , and K include both multiples and non-multiples of 16 and 32.

8.3 Step 2: Add Configuration Parameters

Add generator parameters in Gemmini configuration files:

Listing 6: Example parameter names.

```
mxEnable: Boolean
mxFormats: Seq[MXFormat] = Seq(MXINT8)
mxBlockSize: Int = 32
mxScaleMemKB: Int
mxSupportDim16: Boolean
mxUseFixedPointAccum: Boolean
```

Create two experiment configs:

- `GemminiMXINT8DIM32Config`
- `GemminiMXINT8DIM16Config`

Exit criterion: stock non-MX Gemmini tests still pass with MX disabled.

8.4 Step 3: Add MXScaleSRAM

Modify `Scratchpad.scala` or a nearby memory module to include `MXScaleSRAM`. The first implementation can be a simple SRAM-like structure indexed by a software-visible metadata address. Add:

- metadata write port,
- metadata read port for A scale vector,
- metadata read port for B scale vector,
- optional arbitration if a single dual-port SRAM is used,
- counters for metadata reads and writes.

Exit criterion: baremetal `mx_scale_mvvin_mvout` test passes. This test should load scale bytes into `MXScaleSRAM`, read them back, and compare against the expected vector.

8.5 Step 4: Add Config and Command Interface

Add a `config_mx` path or extend an existing configuration command. The first command should set:

- MX enable bit,
- scale base addresses,
- current format = `MXINT8`,
- fixed-point accumulator exponent policy,
- flags for future transpose-aware modes.

Exit criterion: software can enable/disable MX mode without perturbing legacy int8 tests.

8.6 Step 5: Add MXKBlockController for DIM=32

In the DIM=32 case:

1. issue compute command,
2. fetch A scale vector,
3. fetch B scale vector,
4. latch both vectors,
5. run the normal Gemmini compute episode,
6. send scale vectors to `BlockScaleUnit`,
7. advance logical K-block.

The controller should expose debug signals:

- current logical K-block,
- current metadata addresses,
- current valid K mask,
- scale-vector valid bits,
- stalls caused by metadata unavailability.

Exit criterion: identity-scale mode runs and gives identical results to baseline int8 for K=32.

8.7 Step 6: Add BlockScaleUnit for K=32

Implement `BlockScaleUnit` after the mesh output and before accumulator ingress. The first version should support:

- int32 raw partial input,
- A scale exponent vector,
- B scale exponent vector,
- fixed output exponent,
- shift and saturate,
- row-by-row operation.

Exit criterion: single-block GEMM with K=32 matches the software golden model.

8.8 Step 7: Support K Greater Than 32

Add per-block scaled accumulation:

1. run block b ,
2. produce raw partial tile $P^{(b)}$,
3. scale $P^{(b)}$ using the block's A/B scales,
4. accumulate scaled contribution into the accumulator,
5. advance to block $b + 1$.

Exit criterion: K=64, K=96, and K=128 tests match the golden model.

8.9 Step 8: Implement DIM=16 Two-Phase K-Block Execution

This is the research step. Modify `MXKBlockController` so that one logical MX K-block spans two physical Gemmini K phases:

Listing 7: DIM=16 controller invariant.

```
fetch scales for logical K-block b
run physical half 0 with k lanes 0..15
hold the same scale vectors
run physical half 1 with k lanes 16..31
scale once for the full logical 32-element block
advance b
```

Exit criterion: DIM=16 tests match the golden model and instrumentation proves that metadata advances every 32 logical K lanes, not every 16 physical lanes.

8.10 Step 9: Add Transposer-Aware ScaleVectorGenerator

Start with a software-prepacked mode. Then add hardware metadata reconstruction for transposed logical views. The goal is not to build a scale transposer. The goal is to map logical operand roles to vector indices.

Exit criterion: tests involving explicit transpose flags and OS-implied transposer behavior either pass or are explicitly excluded from the first conference scope. Do not silently claim support for modes not verified.

8.11 Step 10: Integrate Loop-Level Support

Only after the direct baremetal tests pass, modify higher-level loop matmul support to emit MX-aware tiling and metadata movement. The loop engine should tile K in logical 32-element blocks, not raw DIM rows.

Exit criterion: a tiled GEMM larger than one tile runs through the C wrapper, not hand-written instruction sequences.

9 Validation of the Designed Architecture

9.1 Validation Layers

Use a layered validation strategy:

Layer	Tests	Exit criterion
Software unit	encode/decode, scale selection, tail padding	deterministic and random tests pass
Hardware unit	<code>MXScaleSRAM</code> , exponent decode, <code>BlockScaleUnit</code>	module-level tests pass
Tile-level RTL	K=32, K=64, K tail, identity-scale	matches golden model
Transposer cases	normal, explicit transpose, OS-implied transpose if supported	metadata vectors match logical model
System tests	tiled GEMM, microbenchmarks, loop wrappers	output and counters reproducible

9.2 Required Corner Cases

The validation suite must include:

- all-zero blocks,
- maximum positive payloads,
- maximum negative payloads,
- alternating signs,
- outlier-heavy rows,
- outlier-heavy columns,
- K not divisible by 32,
- M and N not divisible by DIM,
- identity-scale mode,
- random scale exponents,
- DIM=32 aligned mode,
- DIM=16 two-phase mode,
- explicit transpose if supported,
- OS-implied transposer if claimed.

9.3 Invariants to Assert

Add assertions or debug checks for:

1. **Metadata validity:** compute cannot start in MX mode until A and B scale vectors are valid.
2. **DIM=16 hold invariant:** scale vectors do not change between half 0 and half 1 of a logical MX block.
3. **K-block advance invariant:** metadata index increments exactly once per 32 logical K lanes.
4. **Tail invariant:** invalid tail lanes contribute zero raw product.
5. **No inline scale path:** scale bytes are never fed into the payload transposer as payload elements.
6. **Block-scale invariant:** each raw partial tile is scaled before being accumulated with a different logical K-block.

9.4 Debugging Strategy

If a mismatch occurs, debug in this order:

1. Dump A and B scale vectors for the failing output tile.
2. Check logical K-block index.
3. Check valid K mask.
4. Check whether DIM=16 half 0 and half 1 used the same scale vectors.
5. Check raw int32 partial before scaling.
6. Check BlockScaleUnit exponent and shift.
7. Only then inspect payload transposer timing.

Most MX bugs will be metadata bugs, not payload bugs. A single wrong scale byte can poison an entire output row or column.

10 FireSim Simulation Plan

10.1 When to Use FireSim

Use Verilator/VCS first for short RTL debug. Move to FIRESIM only when:

- baremetal tests pass in RTL simulation,
- the target config builds cleanly,
- performance counters are inserted,
- benchmark binaries are stable.

FireSim should be used for longer workloads, full-system behavior, and performance-counter collection. It is not the right first debugging environment for a broken scale-vector unit.

10.2 FireSim Setup Checklist

The FireSim/VCU118 flow should follow this sequence:

1. Freeze Chipyard, Gemmini, FireSim, and Vivado versions.
2. Run a stock FireSim VCU118 example to verify the board and host setup.
3. Build a stock Gemmini target and run a known Gemmini workload.
4. Build `GemminiMXINT8DIM32Config`.
5. Run baremetal MX tests under FireSim.
6. Add `AutoCounter` instrumentation.
7. Run microbenchmarks and collect CSV counter output.
8. Build `GemminiMXINT8DIM16Config`.
9. Run DIM=16 baseline and proposed design workloads.

10.3 Counters to Add

Add counters in `ExecuteController`, `Scratchpad.scala`, `MXScaleSRAM`, `MXKBlockController`, `ScaleVectorGenerator`, and `BlockScaleUnit`. At minimum collect:

Metric	Purpose
Kernel cycles	main latency metric
Useful MACs	throughput normalization
Payload DMA bytes	baseline data movement
Metadata DMA bytes	cost of MX side metadata
<code>MXScaleSRAM</code> reads/writes	metadata path activity
Metadata stall cycles	whether scale fetch delays compute
K-block count	validates logical block schedule
DIM=16 half count	validates two-phase mapping
Transposer active cycles	confirms payload path behavior
<code>BlockScaleUnit</code> active rows	confirms block-scale application
Accumulator write count	validates scaled accumulation
Resource usage	LUT, FF, BRAM, DSP, frequency

Use `AutoCounter` for final performance counters. Use `TracerV` only for debugging software control flow or marking regions of interest, not for final throughput claims.

10.4 FireSim Experiments

The minimum FireSim experiment matrix:

Configuration	DIM	MX mode	Purpose
Stock Gemmini int8	32	off	baseline for aligned config

MXINT8 aligned	32	side metadata	verifies working MX path
Stock Gemmini int8	16	off	default-style baseline
Naive MXINT8	16	duplicate/repack	shows integration overhead
Proposed MXINT8	16	logical K-block metadata	demonstrates contribution

Benchmarks should include:

Priority	Workload	Source or shape family	Year-one decision
Must-have	Synthetic GEMM suite	Square, skinny, fat, and K-tail shapes with $K \in \{32, 64, 128, 256, 512\}$	Include
Must-have	Transformer-derived GEMM traces	BERT-style attention and MLP shapes, or MLPerf-derived transformer shapes	Include
Should-have	ResNet50-style operator shapes	MLPerf/ImageNet-style convolution-lowered GEMM shapes	Include if low friction
Should-have	BERT-Large software accuracy sanity	SQuAD-style software-only evaluation	Include in software only
Optional	GPT-J/Llama trace replay	MLPerf language-model shapes, not full deployment	Late-stage optional
Reject	Full DLRM.v2 end to end	Too much system complexity for this project	Exclude
Reject	Full Llama2-70B on Gemmini	Unrealistic for this schedule	Exclude

The workload justification should cite MLPerf/MLCommons as a source of representative inference task families without claiming full MLPerf compliance [20, 21]. The artifact should emit machine-readable CSVs for every run and should include shape files so reviewers can reproduce the experiment matrix.

11 Baselines, Ablations, and Evaluation Discipline

A weak baseline set will kill this paper. The final evaluation should include the following baselines:

Baseline	Definition	Purpose
Stock Gemmini INT8	Normal int8 path with no MX metadata	Lower-bound latency/resource reference
DIM=32 aligned proposed	Proposed metadata engine with natural 32-lane alignment	Separates easy plumbing from DIM=16 research
Software-repack baseline	Software repacks metadata for each operand orientation or transpose case	Realistic software workaround

Duplicate-transposed metadata baseline	Store extra metadata views to avoid run-time remapping	Realistic storage/-movement trade-off
Proposed DIM=16 logical-K design	Main contribution	Tests logical/physical mismatch solution
Oracle no-stall variant	Proposed design with metadata stalls removed in simulation/counters	Quantifies headroom

The following ablations are required:

- DIM=32 versus DIM=16;
- metadata stall path enabled versus oracle no-stall;
- duplicate metadata versus logical remapping;
- int32 versus 48/64-bit shadow accumulation in the golden model and, if feasible, RTL;
- tail-block behavior for odd K;
- identity-scale mode, which must collapse to stock INT8 semantics;
- transpose-aware path enabled versus disabled, if the paper claims transposer-aware support.

Report cycles, useful MACs, payload bytes, metadata bytes, metadata stall cycles, scale-SRAM accesses, logical K-block transitions, transposer activity, `BlockScaleUnit` activity, LUTs, FFs, BRAMs, DSPs, and Fmax. Throughput-only plots are insufficient.

12 Paper Impact

12.1 What the Paper Contributes

The paper should claim four contributions:

1. **Problem characterization:** MX metadata interacts badly with transposer-based systolic arrays when physical array width does not match the 32-element MX block size.
2. **Architecture:** a sidecar metadata execution mechanism with `MXScaleSRAM`, `MXKBlockController`, `ScaleVectorGenerator`, and `BlockScaleUnit`.
3. **Implementation:** a full Gemmini integration, not a standalone MAC block.
4. **Evaluation:** correctness, area/timing overhead, metadata traffic, and performance versus naive duplicate/repack baselines.

12.2 What Not to Claim

Do not claim:

- state-of-the-art quantization accuracy,
- general support for all MX formats,
- training support,

- full arbitrary-DIM support unless implemented,
- OS and WS support unless both are tested,
- model-level wins without measuring end-to-end overheads.

12.3 Paper Figures

The paper should have these figures:

1. MX block and scale grouping diagram.
2. DIM=16 mismatch: one MX block spans two Gemmini K phases.
3. Naive duplicate/repack baseline diagram.
4. Proposed sidecar metadata architecture.
5. DIM=32 execution timeline.
6. DIM=16 two-phase logical K-block timeline.
7. Performance chart: stock vs naive vs proposed.
8. Metadata traffic chart.
9. Area/timing delta chart.

12.4 Paper Tables

The paper should have these tables:

1. Supported MX policy and numerical semantics.
2. Gemmini configurations.
3. Hardware module changes.
4. Validation corner cases.
5. Counter list.
6. Resource overhead.
7. Main ablation results.

13 Risk Management

13.1 Critical Weaknesses to Fix Before Submission

The current plan is technically promising but will be vulnerable in review unless the following weaknesses are fixed:

Weakness	Why it hurts	Required fix
Implementation-centric framing	“Added MXINT8 to Gemmini” sounds incremental	Frame as metadata execution under logical/physical K mismatch
No formal proof obligation	Reviewers cannot tell what correctness means	Use the invariant in this document as the paper anchor
Accumulator semantics ambiguous	$K > 32$ can silently produce wrong results	Freeze exponent policy and accumulator width
Transposer claim too broad	WS-only evidence cannot support a transposer-wide claim	Validate at least one OS/-transpose case or narrow the claim
Baselines underdeveloped	Bad baselines make architecture papers unconvincing	Include software-repack, duplicate-metadata, DIM=32, and oracle baselines
Workloads vague	“Transformer-like” is not an experiment	Name and freeze shape sources
No model-level sanity	Reviewers distrust new datatype systems without accuracy sanity	Add software-only accuracy deltas for at least one CNN and one transformer-derived workload
No area/timing method	Cycles alone are not hardware evidence	Include FPGA resource/F-max and, if possible, ASIC-style synthesis estimates

13.2 Main Risks

Risk	Why it matters	Mitigation
Accumulator semantics become messy	$K > 32$ requires per-block scaling before accumulation	Start $K=32$, then fixed-point accumulation; document exponent policy
DIM=16 control is too invasive	This is the real contribution but hardest logic	Keep DIM=32 always working; add DIM=16 only after MVP
Transposer modes explode scope	OS/transpose interaction can derail schedule	Start WS-only; claim only tested modes
FireSim version drift	Chipyard/Gemmini/FireSim stacks change	Freeze commits in month 1
Reviewers see a Gemmini hack	Rejection risk	Present general pattern for transposer-based systolic arrays

Benchmarks sprawl	Two people cannot do everything	Precommit to microbenchmarks plus one realistic workload
-------------------	---------------------------------	--

13.3 Stop/Go Criteria

Time	Go criterion	If not met
End month 1	Golden model and MX policy frozen	Stop RTL; fix semantics
End month 2	Metadata SRAM round-trip works	Do not start <code>BlockScaleUnit</code> yet
End month 3	DIM=32 identity-scale compute passes	Debug control path before scaling
End month 4	DIM=32 scaled K=32 GEMM passes	Simplify accumulator policy
End month 5	K>32 scaled accumulation passes	Limit first paper to smaller scope or workshop
End month 6	DIM=16 naive/proposed comparison visible	Continue to conference if data is compelling

14 Two-Person Work Plan

Area	Person A: RTL/Architecture	Person B: Software/Evaluation
Month 1	Read Gemmini modules; design <code>MXScaleSRAM</code>	Golden model; quantization policy; packer
Month 2	Implement <code>MXScaleSRAM</code> and config path	Metadata round-trip tests; C wrappers
Month 3	Implement <code>MXKBlockController</code> DIM=32	Identity-scale tests; random test vectors
Month 4	Implement <code>BlockScaleUnit</code> and K>32 accumulation	Golden comparison; plotting scripts
Month 5	FireSim config and counters	Benchmark harness; CSV parser
Month 6	DIM=16 controller and scale-vector generation	Naive baseline; paper figures
Month 7+	Timing/resource cleanup	Artifact packaging; paper writing

Both people should jointly review failing tests. Metadata bugs often look like arithmetic bugs, so the software and RTL sides cannot be isolated.

15 Publication Roadmap and Milestones

The schedule should be managed as a conference-first project with a hard six-month design target. The design should not remain exploratory past the first month.

Window	Owner	Milestone	Exit criterion
--------	-------	-----------	----------------

May 2026	Both	Semantics freeze	policy file, exact rounding/range/-tail rules, workload list frozen
June 2026	Software lead	Golden model and packer	randomized reference tests pass on odd/even M,N,K
July 2026	RTL lead	MXScaleSRAM and load/store path	metadata round-trip test passes
August 2026	RTL lead	DIM=32 identity and scaled path	exact match to stock INT8 in identity mode and golden model in scaled mode
September 2026	Both	K>32 accumulation	K=64/96/128 exactness under frozen policy
October 2026	RTL lead	DIM=16 logical-K controller	two-phase hold invariant proven by tests and counters
November 2026	Both	First evaluation cut	baseline/proposed plots and resource/Fmax reports exist
December 2026	Both	Scope review	choose short letter fallback or full conference paper
Jan–Feb 2027	Both	Mature artifact	scripts, CSVs, configs, commit hashes, and reports reproducible
March 2027	Both	Conference submission package	paper, appendix, artifact, and rebuttal notes ready

The most realistic primary venue is an IEEE/ACM reconfigurable-computing or architecture venue such as FCCM if the DIM=16 result is strong and the artifact is real [22, 23]. A journal version should not be the first target. It should follow only after the work adds broader dataflow support, stronger synthesis evidence, and expanded workloads.

16 Recommended Final Deliverables

1. `mxint8_policy.md`: exact numerical policy.
2. `mxint8_golden.py` or C++ equivalent.

3. Gemmini fork or patch set with MXINT8 support.

4. Baremetal tests:

- `mx_scale_mvin_mvout`,
- `mx_identity_gemm`,
- `mx_k32_gemm`,
- `mx_k64_gemm`,
- `mx_dim16_two_phase`.

5. FireSim configs and workloads.

6. AutoCounter CSV logs.

7. Vivado resource and timing reports.

8. Reproduction scripts.

9. Paper figures and tables generated from scripts.

17 One-Page Execution Checklist

17.1 Before More RTL

- Freeze one numerical contract: payload range, rounding, saturation, zero-block rule, and tail-block rule.
- Freeze one paper claim centered on metadata execution under logical/physical K mismatch.
- Freeze the workload list and explicit exclusions.
- Freeze repository commits, tool versions, seeds, and container environment.
- Write the formal correctness invariant into both the design document and the paper draft.

17.2 Hardware Must-Haves

- Sidecar metadata SRAM with counters.
- Logical K-block controller with explicit DIM=16 hold invariant.
- Exponent decode moved off the hot path.
- Wide shadow accumulator selected and documented.
- Identity-scale mode exactly matches stock INT8 behavior.
- Tail masking formally tested.
- At least one transpose-aware or OS-related validation case exists, or the final claim is narrowed.

17.3 Evaluation Must-Haves

- Stock INT8 baseline.
- Software-repack baseline.
- Duplicate-transposed-metadata baseline.
- DIM=32 proposed design.
- DIM=16 proposed design.
- Oracle no-stall comparison.
- Resource and Fmax reports.
- Metadata bytes, metadata stalls, transposer activity, and total cycles all reported.

18 Final Recommendation

The first successful version should be deliberately narrow:

Item	First target
Format	MXINT8 only
Payload arithmetic	$\text{int8} \times \text{int8} \rightarrow \text{int32}$
Dataflow	weight-stationary first
Bring-up geometry	DIM=32
Research geometry	DIM=16
Metadata architecture	sidecar MXScaleSRAM
Core controller	logical 32-element K-block tracker
Scale application	block boundary, before cross-block accumulation
Main baseline	duplicate/repack or transpose-time requantization
Paper claim	metadata execution for transposer-based systolic arrays

If this plan succeeds, the conference paper is strong. The journal paper can then add broader dataflow support, more workloads, stronger synthesis data, and possibly MXFP8 or packed MXFP4/MXFP6 as follow-on extensions. Do not add those before the metadata execution mechanism is correct and measured.

References

- [1] Open Compute Project, *OCP Microscaling Formats (MX) Specification Version 1.0*, Sept. 2023. Available: <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>
- [2] H. Genc et al., “Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration,” arXiv:1911.09925, 2019. Available: <https://arxiv.org/abs/1911.09925>
- [3] UCB-BAR, *Gemmini: Berkeley’s Spatial Array Generator*, GitHub repository. Available: <https://github.com/ucb-bar/gemmini>

- [4] FireSim Documentation, *AutoCounter: Profiling with Out-of-Band Performance Counter Collection*. Available: <https://docs.firesim/en/latest/Advanced-Usage/Debugging-and-Profiling-on-FPGA/AutoCounter.html>
- [5] FireSim Documentation, *On-Premises FPGA Getting Started: Xilinx VCU118 FPGAs*. Available: <https://docs.firesim/en/t/Getting-Started-Guides/On-Premises-FPGA-Getting-Started/Xilinx-VCU118-FPGAs.html>
- [6] NVIDIA, *NVIDIA Tensor Cores*. Available: <https://www.nvidia.com/en-eu/data-center/tensor-cores/>
- [7] NVIDIA, *Transformer Engine User Guide: FP8 Primer and Microscaling Examples*. Available: https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html
- [8] NVIDIA, *cuDNN Backend Release Notes*. Available: <https://docs.nvidia.com/deeplearning/cudnn/backend/v9.21.1/release-notes.html>
- [9] AMD, *AMD Instinct MI350 Series GPUs*. Available: <https://www.amd.com/en/products/accelerators/instinct/mi350.html>
- [10] AMD, *Microscaling (MX) Quantization, AMD Quark Documentation*. Available: https://quark.docs.amd.com/latest/onnx/tutorial_microscaling_quantization.html
- [11] AMD, *Vitis AI Documentation*. Available: <https://vitisai.docs.amd.com/>
- [12] Intel, *Agilex FPGAs Deliver a Game-Changing Combination of Capabilities*, white paper. Available: <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/agilex-fpgas-game-changing-white-paper.pdf>
- [13] U. Koster et al., “Flexpoint: An Adaptive Numerical Format for Efficient Training of Deep Neural Networks,” arXiv:1711.02213, 2017. Available: <https://arxiv.org/abs/1711.02213>
- [14] D. Drumond et al., “Training DNNs with Hybrid Block Floating Point,” arXiv:1804.01526, 2018. Available: <https://arxiv.org/abs/1804.01526>
- [15] H. Sharma et al., “Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Networks,” arXiv:1712.01507, 2017. Available: <https://arxiv.org/abs/1712.01507>
- [16] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks,” IEEE ISSCC, 2016. Available: https://dspace.mit.edu/bitstream/handle/1721.1/101151/eyeriss_isscc_2016.pdf
- [17] MicroScopiQ authors, “MicroScopiQ: Accelerating Microscaling Quantized Neural Networks,” arXiv:2411.05282, 2024. Available: <https://arxiv.org/abs/2411.05282>
- [18] M²XFP authors, “M²XFP: Efficient Microscaling Floating-Point Processing,” arXiv, 2025. Available: <https://arxiv.org/html/2505.22404v1>
- [19] BBAL authors, “BBAL: Block-Based Arithmetic for Low-Precision AI Acceleration,” DAC, 2025. Available: <https://dl.acm.org/doi/10.1109/DAC63849.2025.11132978>

- [20] MLCommons, *MLPerf Inference Benchmarks Documentation*. Available: <https://docs.mlcommons.org/inference/>
- [21] MLCommons, *MLPerf Inference v6.0 Results*, Apr. 2026. Available: <https://mlcommons.org/2026/04/mlperf-inference-v6-0-results/>
- [22] IEEE/ACM FCCM, *Call for Papers*. Available: <https://www.fccm.org/call-for-papers-2026/>
- [23] IEEE/ACM FCCM, *Artifact Evaluation*. Available: <https://www.fccm.org/artifact-evaluation-2026/>
- [24] Critical review generated for this project, *Critical Review of the MXINT8 Gemmini Research Plan*, Apr. 2026.